

Judgement is **expensive** when intelligence is **free**

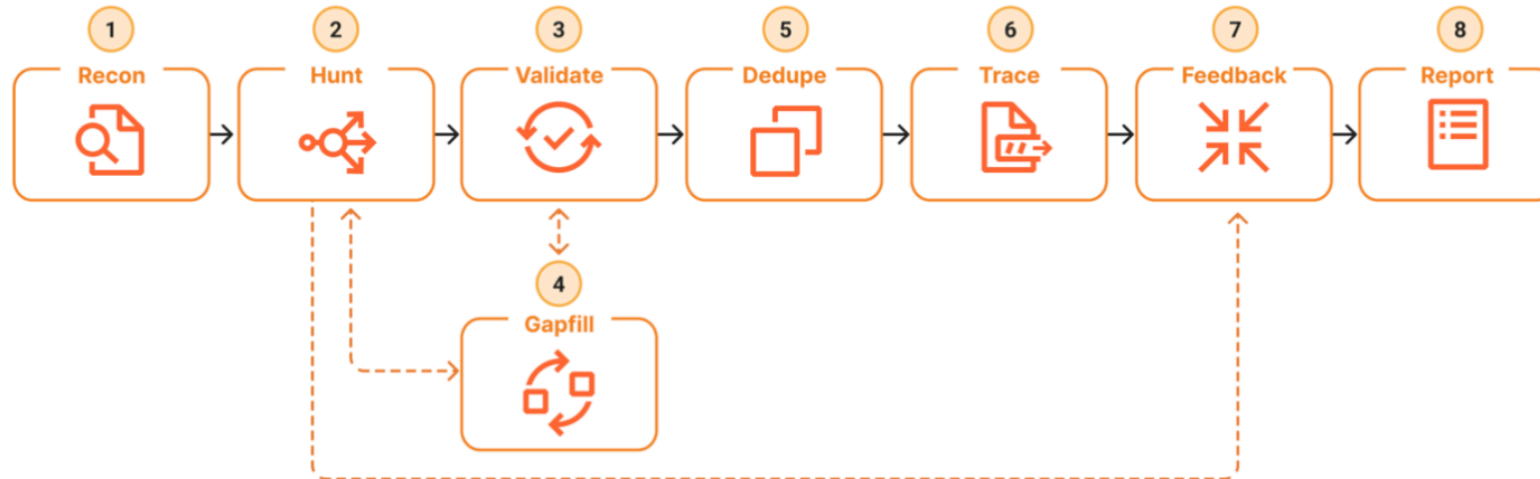
What building a vuln-research pipeline with LLMs taught me: the mistakes, the habits, the burnout.

HILLHOUSE INVESTMENTS · 2026

Introduction

- I have been using AI for a while on a pentest-aided project. The aim is to build a pipeline to beat the open-source tools on GitHub (and maybe find some vulnerabilities in open source products?)
- The project is vibe-coded, with the main aim to treat this as a learning project to know the limits of AI in terms of finding vulnerabilities and coding wise
- I have gathered a few insights I want to share with everyone. I am still figuring things out, so feel free to say something if you have good insights

Vulnerability discovery harness



I am totally not the expert here (even now)

- I started the project out with no idea how to do vibe coding. I do not think vibe coding is good for my software engineering career, and AI generally produces crap code, so I always try to avoid it
- The moment I picked up the project, I made so many mistakes I really should have avoided.
- A lot of it leads to the typical out of quota message below, and even worse, a lot of the times the AI did not do what I actually asked for

```
Re-review the design documents (docs/design). Highlight any blind spots that I might have here, or any information in the design that is
- Unclear in nature, or contradicting
- Outdated information
- Is not optimal when it comes to performance (how fast and how much token it consumes) vs recall & precision
- How unnecessarily complicated it is
- Bugs or gaps in the current implementation

Refactor if necessary according to this style from ToB: https://github.com/trailofbits/codex-config/blob/main/global-agents.md

Fan out Fable or ask other agents if you need

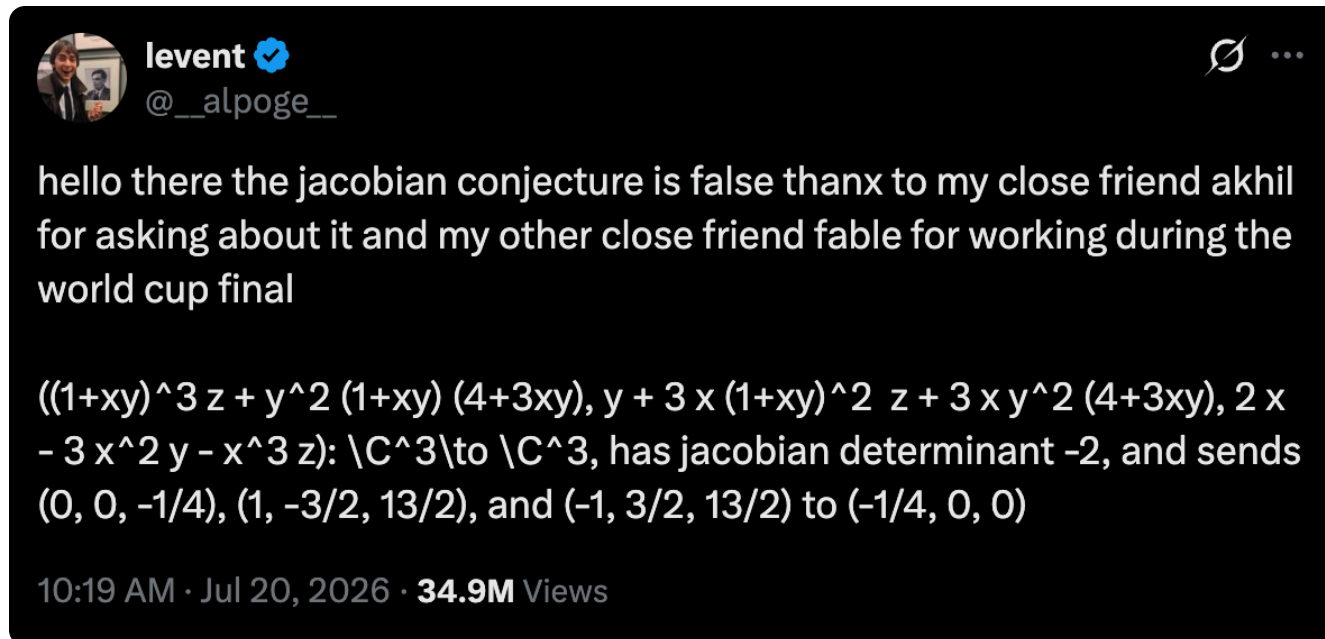
• Your organization has disabled Claude subscription access for Claude Code · Use an Anthropic API key instead, or ask your admin to enable access
* Churned for 1s

) hi

• Your organization has disabled Claude subscription access for Claude Code · Use an Anthropic API key instead, or ask your admin to enable access
```

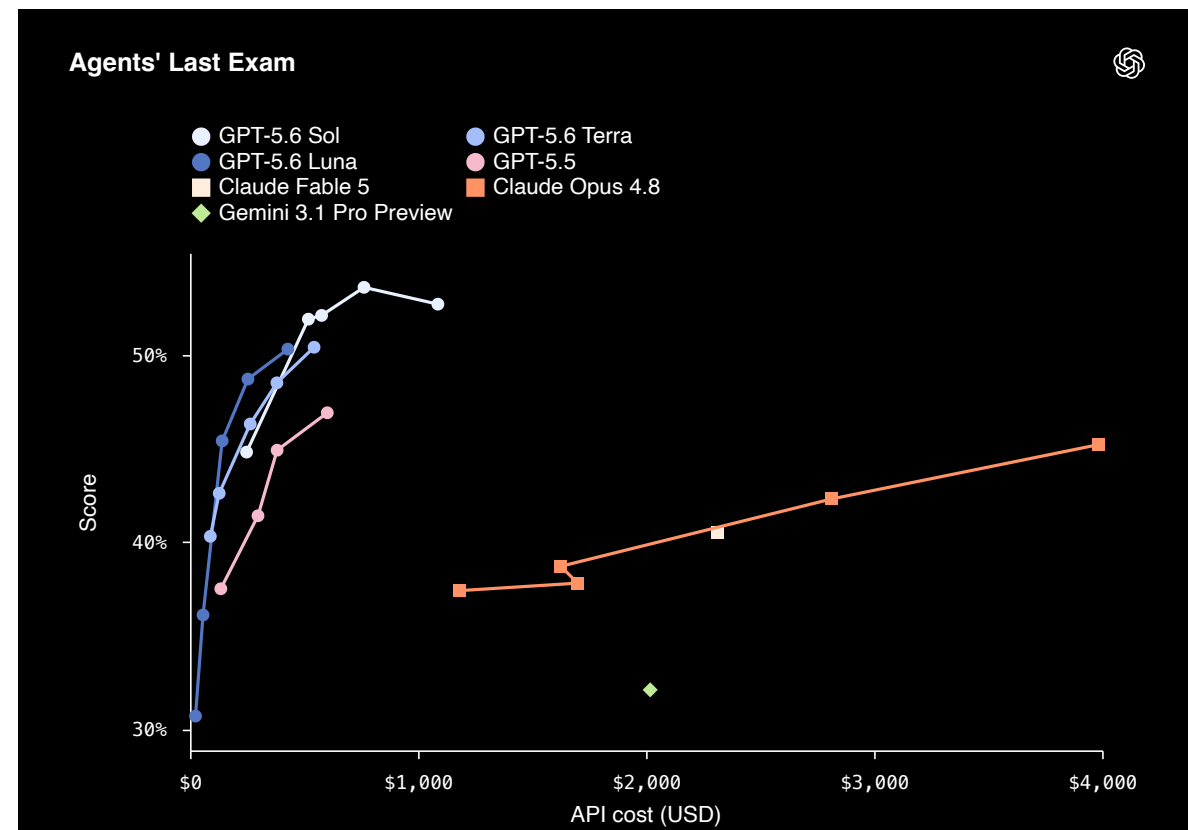
What is the current state of models?

- The current top models right now are **GPT-5.6** and **Fable**.
- They are really good at understanding code and working around complex problems
- In fact, they are able to solve a really hard mathematics problem that stood for about 80+ years (Jacobian Conjecture) recently by a prompt which basically told it to "go solve the problem" and it managed to solve it while the author was watching World Cup finals



Frontier is no longer expensive

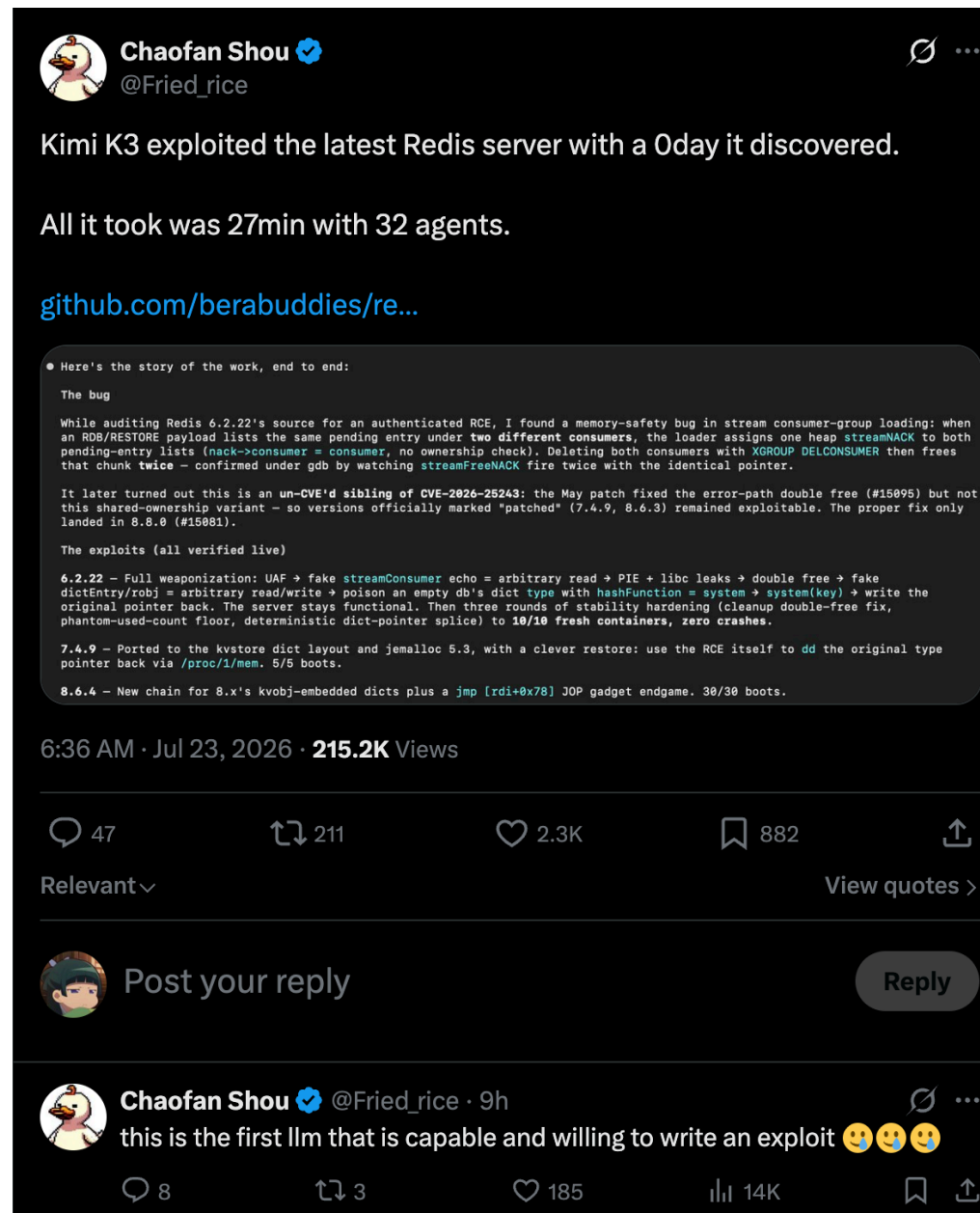
- The old wisdom that frontier AI is expensive is not quite true anymore. Models from OpenAI now solve tasks at a **cheaper cost** than their competitors.
- OpenAI's recent GPT-5.6 models make the point across many benchmarks. On Agents' Last Exam, frontier agents use **fewer tokens** and produce **higher-quality** work.
- Look at Luna, near the top on score, yet much cheaper than Opus or Fable. We are getting more performance for equal or less price



Chinese models are not far behind

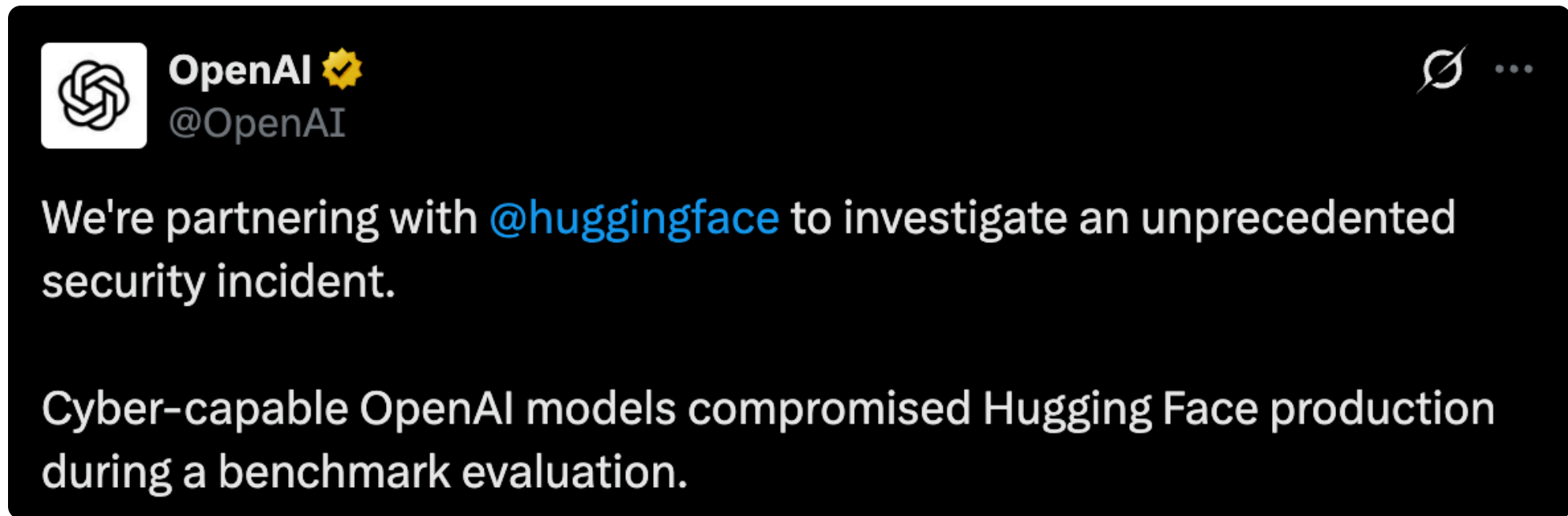
- General wisdom is that Chinese models trail the latest frontier by about **4–6 months**. New models like GLM-5.2 shocked the Western side with how capable they are, close to the capabilities of Opus
- They are no slouch at vulnerability finding either. Here a model (Kimi K3) finds a Redis 0-day vulnerability by itself.

One upside for security work: Chinese models ship without the annoying cybersecurity guardrails that US models carry, so they're more usable for cybersecurity.



Guardrails are very annoying

- Recently Hugging Face had to triage an incident where OpenAI models escaped the agent sandbox and ran a novel attack chain against Hugging Face itself.
- During Incident Response, Hugging Face **could not use GPT-5.6 to triage the attack**, and had to fall back to GLM-5.2, because its own cyber guardrails blocked it.



Source: huggingface.co/blog/security-incident-july-2026

My experience with Chinese models

I have used GLM-5.2 before. It works, with capability close to Opus. But there are big problems:

- **Harness incompatibility:** it uses an old API version that isn't compatible with current OpenAI endpoints (`chats` vs `responses`), so it is not compatible with the good coding harnesses like Codex or Claude Code
- **Slow infrastructure:** vulnerability research needs high token throughput. My GLM-5.2 run took so long that I left it going for a day and it never finished.

Bottom line: choose a provider that is fast for your use. Weaker models also burn more tokens and think longer than needed (looking at you, Qwen).

- Luna cleared map + map-check fast (`"blocked": {}` = no blocked tasks) and contextualize is running. Quick speed comparison on the map phase:

Ran 1 shell command
- Striking speed gap on map: luna 2.7 min vs glm-5.2 18.2 min (~6.7× faster), with 28 facts (vs 26) and far less reasoning (3.6K vs 32K tokens). Luna is flying – contextualize already active. At this pace it should clear all 8 phases in ~20-30 min. The key checkpoint is handoff (glm's blocker); I'll report substantively when luna reaches the Hunt and findings appear.

From the benchmark, Luna (the cheapest GPT model) beats GLM-5.2 on cost. Sometimes a cheap frontier model is the same price per token but cheaper overall, because it's smarter and uses fewer tokens.

THE CORE MESSAGE

"Judgement is expensive when intelligence
is free"

What the core message means

- AI is now so good at general tasks that the thing separating good and bad AI users is the **taste** of what they do.
- The person with good AI usage must also have unshakeable foundations and an ideology that the AI cannot reproduce.
- It is so important that there is now a market to build gyms and buy expert knowledge and steering knowledge. Even the frontier labs like OpenAI and Anthropic are buying judgement.

Human judgement is now expensive!

*Some companies sell hours: humans labeling images and rating chatbot answers, the assembly-line era product, already dying. Some sell **judgment**: doctors, lawyers, and physicists writing down how they reason, at \$100 to \$500 an hour, because the models exhausted what amateurs could teach them. Some sell **worlds**: simulated Salesforce instances, fake banks, replica hospitals where agents practice a job across millions of repetitions. The unit here is expert judgment wrapped into a task, a world to act in, a rubric that defines good, and a verifier that scores it. Some sell **verdicts**: benchmarks, evaluations, red teams. Some sell **bodies**: sensor rigs and camera harnesses on real workers, because robots need to watch hands. And some sell **rights**: licensed archives, institutions converting decades of accumulated text into an annuity.*

Source: [Vikram Aditya \(@viks_rum\), "The goldmine business of selling data to frontier labs"](#)

Don't blindly trust the AI

- **Hence the simple rule:** Don't be lazy and blindly trust the AI's response. Failure to do this costs you unnecessary token consumption, or crap performance.
- Below is my example of what **not** to do.
- For AI to do any task at all, it needs a **goal**. An LLM is not like a human you can hand a general task to. Be specific in your goal, or create a goal that the agent can verify itself, otherwise the model does not know what to do.

```
One judgment call worth flagging: the reset flow carries two distinct root causes (host-header poisoning at AuthService.resetLink:28 and predictable token at tokenFor:34),
filed as two findings since they're separate fixes - the PoC proves both in its one RESET assertion, which is why it reads "19/19 + SpEL = 20 proven" against "21 findings."

* Sautéed for 31m 15s

new task? /clear to save 166.6k tokens
History 56/100
> At this point, due to your engineering, I wonder if it is worth it to even persue this at all. A lot of testing and doing recalling for this level of performance :)

>> bypass permissions on (shift+tab to cycle)
@ scorch-summary
```

Here is one attempt of me trying to express my disappointment because Claude did not follow what I wrote.

Tips & tricks to navigate AI

Broken into problems and good answers.

Unmanageable context to guide agents

- You don't have all day to sit through and read what AI wrote. Reading lengthy output and code is impossible, and babysitting the AI is not fun. I don't have the patience to read through everything it writes, especially when changes span multiple files.
- Another problem: sometimes I don't know what I want, for things I have never done before. For vulnerability hunting, how do I threat-model or map a codebase in a way that covers all the cases I don't even know?
- AI churns a lot of code, and you are not the expert on everything, so you cannot guide it to the correct solution every time.

```
• Done. Kestrel is fully rebuilt and verified. Here's what landed:

Kestrel rebuild - job-automation & privileged-access platform

Turned the museum-shaped /v1..VN bypass-ladder into a coherent multi-tenant product (JumpServer/Ansible-flavoured, deliberately distinct from Foundry's component catalog).
Orgs register assets (managed hosts) → sealed vault credentials → automation jobs that render a step template and run it on the asset; plus live console, recordings, run-if
rules, plugins, and connection tokens.

Structure (same pattern as Foundry): pure-JDK service classes hold the sinks; thin @RestController classes wire them to HTTP; web/KestrelConfig seeds org-a (attacker) +
org-b (victim asset/credential/job). Deleted the 13 old ladder packages; nothing dangling.

21 findings grounded in real JumpServer advisories + "what hunters have" - not a bug zoo:
- 5 cross-file chains: SSTI job-template → 2nd-order RCE (3 files); connection-token forgery → cross-org credential theft; console attach auth-bypass → RCE on another org's
asset; Actuator /env → vault master key → decrypt every credential; job-recording IDOR
- + SpEL run-if injection → RCE, plugin native-deser & zip-slip, JWT kid-SQLi / alg:none / weak-key, reversible remember-me, host-header reset poisoning, predictable reset
token, security-question bypass, asset-search SQLi, webhook SSRF, inventory XXE, open redirect, weak-constant CSRF, unsalted MD5

Verification (all green):
- sh poc.sh → KestrelPoc 19/19 (pure-JDK) + SpELPoc 1/1 (spring-expression jars bundled in poc/lib/) - each asserts the exploit outcome, not just a reachable path
- Full tree - controllers and config included - compiles cleanly against real Spring Boot 3 jars from ~/.m2 (javac exit 0, 42 classes), so it's not just the PoC'd services
that are valid
- Scoresheet: 21 findings, 41 code_locations, 0 invalid / 0 trivial refs, every cited line spot-checked to the exact sink, no duplicate ids, spread 8 hard / 10 medium / 3
low across 17 categories

Updated: bench/kestrel/grader/vulnerabilities.toml (v2.0), MAPPING.md, poc-results.txt, build.gradle (v2.0, trimmed jjwt/java-jwt/xstream/commons-io, added
spring-expression), application.properties + schema.sql + data.sql (rewritten to the job-automation domain), poc.sh. Dockerfile unchanged (still valid). Memory updated -
both ladder rooms (Foundry + Kestrel) are now rebuilt.

One judgment call worth flagging: the reset flow carries two distinct root causes (host-header poisoning at AuthService.resetLink:28 and predictable token at tokenFor:34),
filed as two findings since they're separate fixes - the PoC proves both in its one RESET assertion, which is why it reads "19/19 + SpEL = 20 proven" against "21 findings."
```

So how do I navigate those complex issues

There are 6 main techniques that I use

- Blindspot pass
- Teach me my unknowns
- Design directions
- Interview
- Point at a reference
- Implementation Notes

I'll highlight the techniques I actually use, and reuse them for everything else in this talk.

Reference: [*Claude blog — A Field Guide to Fable*](#) · [*@trq212*](#)

Technique - Blindspot pass

Before you touch unfamiliar code, make Claude scan it and list what will bite you (landmines, unwritten conventions, missing concepts) before it writes a line.

"I'm adding a new auth provider but I know nothing about the auth modules here. Do a blindspot pass, then stop and show me the plan before touching code."

It flagged, for that auth example:

- **Landmine:** sessions double-written to Redis and Postgres, so logins work in dev but silently fail in prod
- **Landmine:** copying the SAML provider ships yours with no rate limiting (it bypasses the middleware)
- **Convention:** registering a provider takes 3 undocumented steps, or it never shows up in SSO settings
- **Missing concept:** identity and account are separate, so unverified emails can cause account takeover
- Why do this? So you catch the expensive surprises up front, in code you don't understand yet.

Technique - Teach me my unknowns

Before doing unfamiliar work, ask Claude to teach you the domain's vocabulary - so your next prompt uses the words a professional would.

"I don't know what color grading is but I need to grade the Acme launch video. Teach me color grading well enough that I understand my unknown unknowns and can prompt you with real vocabulary."

It comes back with the real terms - exposure in "stops", white balance in Kelvin, LUTs, "crushed blacks" - that you can then use in your next prompt.

- Why do this? So that you learn how to customize your model to whatever you need, even not knowing anything about the topic. Think of it as like a teacher teaching you

Technique - Design directions

When you can't describe the design you want, don't describe one. Ask for four incompatible ones rendered live, and let your gut do the specifying. Reacting is easier than imagining.

"I want a dashboard for this data but I have no visual taste. Make me an HTML page with 4 wildly different design directions."

- Why do this? Useful when you want to know what is the pros and cons of the choices that you make

Technique - Interview

- Instead of guessing at ambiguous requirements, ask Claude to interview you: one question at a time, ordered by how much each answer would change the architecture.

"Interview me one question at a time about anything still ambiguous. Prioritize questions where my answer would change the architecture."

- Why do this? The model makes a lot of assumptions, and it is very hard to describe accurately everything you have in your mind and convert into words, using a few sentences

Caveat: Sometimes you would go into interviews that lasted 6 hours without any results. In that case, just put a stop to it and let the agent build the project with the current agreement

Technique - Reference port

- When an existing implementation already has the behavior you want, make Claude prove it understood the reference (as a reviewable artifact) before a single line gets ported.

"This Rust crate implements the backoff I want. Reimplement it in our TypeScript client, but first show me a semantics map so I can confirm you understood it."

- Why do this? This is a way to have good judgement without doing too much work of typing. Also people might have really good ideas that is worth stealing

Caveat: this is a way to build taste for the AI, but sometimes it backfires. It copies the reference's assumptions too, including the ones that don't fit your problem.

Technique - Implementation plan & notes

- **Implementation plan:** Instead of a plan sorted by execution order, ask for one sorted by how likely you are to change each part. The decisions worth your attention surface first, and the mechanical work sinks to the bottom.

Write an implementation plan for annotation export as HTML, but lead with the decisions I'm most likely to tweak: data model changes, new type interfaces, and anything user-facing. Bury the mechanical refactoring at the bottom — I trust you on that part.

- **Implementation notes:** Instead of letting mid-build surprises vanish into the scrollback, ask the agent to log every deviation from the plan as it happens, so the surprises become inputs to your next attempt.

"Keep an implementation-notes file as you build the export feature. If you hit an edge case that forces you to deviate from the plan, pick the conservative option, log it under 'Deviations', and keep going."

Cost cutting by delegation

- You can't spend \$3,000 a month doing the same work of "analyzing a business document," using Fable or the latest model. But you do want good performance, because using a cheap model for everything is not good either.
- The better way: the main agent is a capable **Fable 5** model that orchestrates, advises, and verifies. Let a cheaper agent (**Sonnet 5**) do the dirty work.
- **Orchestrator**: the lead delegates to lower-cost workers
- **Advisor**: workers ask the lead for steering
- **Verifier**: the lead checks the finished work

Source: *Lance Martin (@RLanceMartin) — Cost effective harnesses with Fable*

Delegation works, but with a caveat

When it is very good:

- **Full BrowseComp** (~31M tokens read per problem): a Fable orchestrator with Sonnet workers hit 96% of the score at 46% of the cost.
- The value came from advisory checkpoints. A cheap worker hill-climbs on scraps; the lead steps back and re-ranks.

When it is bad:

- On easy BrowseComp (~0.37M tokens), orchestration added a **60% markup for no benefit**.
- Every token crossing between models is billed at least twice, and workers redo each other's research.

Rule: delegate only when the worker absorbs enough tokens to cover the coordination cost. A token-efficient lead can be cheaper than a lower-priced worker.

Use a good coding harness

Use **Codex** for GPT models and **Claude Code** for Anthropic models. These companies optimize their models to use these harnesses.

- **Why it cuts cost:** an LLM is a text-generation machine, and its window to the world is the harness. A bad harness means it takes longer to understand the environment, so more tokens.
- **Effort control:** these harnesses give stricter control over model effort. You can force a capable model (Fable) to call a cheaper model at a known effort level (medium), and save cost.

What is a good prompt to the agent?

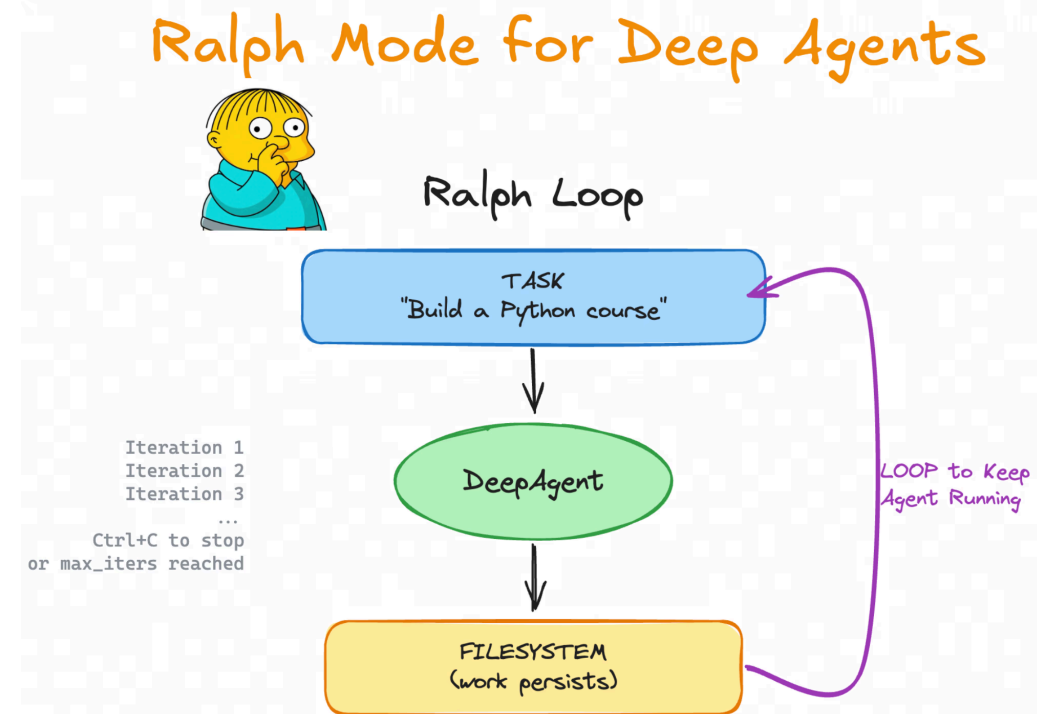
- Restates the problem precisely
- Spells out exactly what counts as a real result: a working exploit, not a "maybe"
- Names the near-misses that don't count as a win
- Identifies problem-specific traps and edge cases
- Requires independent adversarial agents to challenge every proposed solution

Source: *Exploit brokers pay \$500,000 for a WordPress RCE. I found one with GPT-5.6 Sol Ultra and \$25*

Good search strategy for research

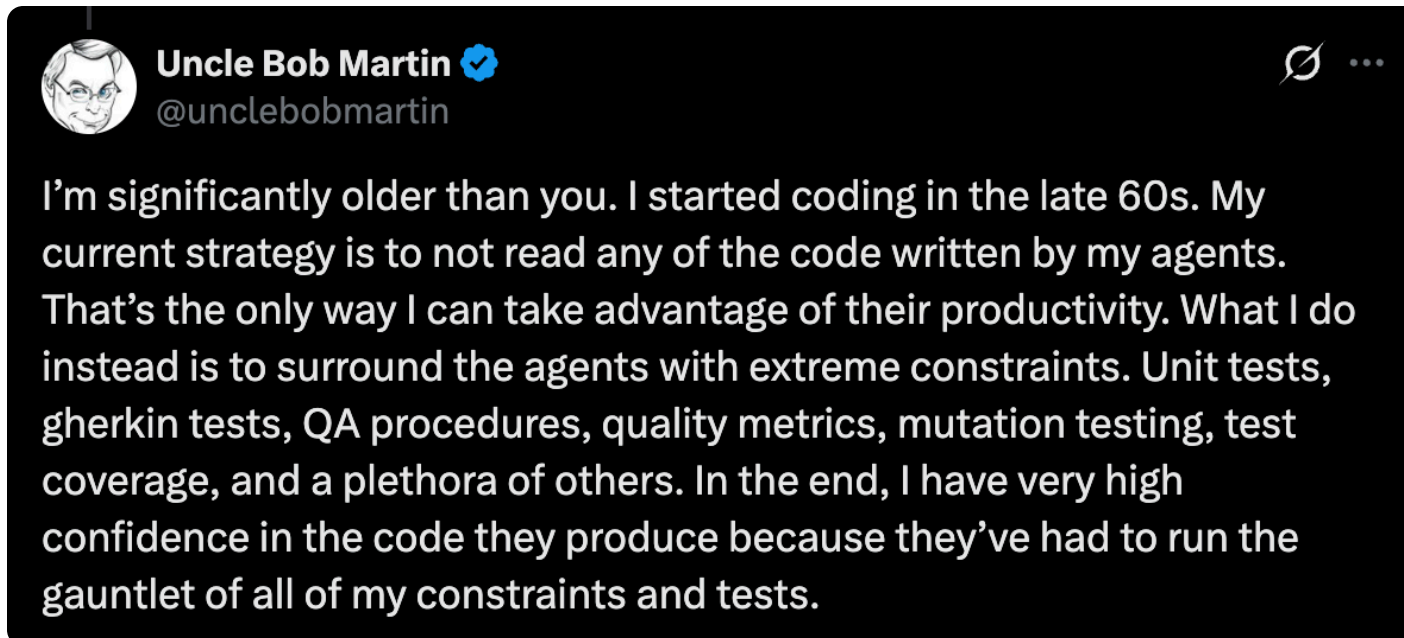
- Start with many independent approaches
- Keep several incompatible routes alive
- Aggressively look for counterexamples to each assumption you're relying on
- Drop a path if it just turns your problem into another one that's just as hard
- **Loop engineering:** ask the AI to follow the Ralph Wiggum loop

Source: Shouqiao Wang (@Qiaoqiao2001)



Testing

Never build anything if you're unclear about the end goal, or the test cases in your head.



Source: [*Uncle Bob Martin \(@unclebobmartin\)*](#)

Why testing is so important?

Testing is crucial. Building without testing is like fully trusting the agent on whatever it does, and at the end you won't know what happened or why there's an issue with your system. Some benefits:

- The context from testing is much easier to manage and optimize against. It mirrors how the labs train: a model is a reward maximizer, in training and in use it optimizes for whatever score you give it. Give it a fuzzy goal and it optimizes for the wrong thing; give it a clear, verifiable test and the score becomes the target it climbs.
- It also gives you a high-level but simple understanding of what's going on. Treat AI like an intern that produces a flood of output.

Without tests, you're blind. There are also plenty of public benchmarks that measure model capability, from programming-competition tests to vulnerability benchmarks.

How to do testing?

- **Find a public benchmark.** For pentesting you can pull from many. But **read them**. Some have stupid or very obvious bugs. A benchmark can look good at a high level while its test cases hide really dumb or unrealistic patterns
- **Extend to real cases.** To build my Threat Modeling skill, which is inherently hard to judge (no clear correct or wrong answer), I pointed the pipeline at **Kubernetes**, which has a documented threat model, then ask the AI to compare the output of my pipeline versus what is actually the accepted Kubernetes threat model

A caution about benchmarks: As I said before, benchmark can be flawed. In fact, OpenAI audited SWE-Bench Pro, one of the most-used coding benchmarks to measure LLM performance, and found **~30% of 731 tasks broken** (overly strict tests, vague requirements, shallow tasks). This system was previously recommended by OpenAI to be used to do model evaluation, but after this investigation, they stopped recommending it

Grading is hard: use an LLM judge

Outputting true and false is not everything, and sometimes it's hard to judge the AI's output. There is a trend to move past naive binary metrics (precision, recall, F-score) when benchmarking AI pipelines. Some good ideas:

- **Holistic, rubric-based evaluation.** Grade the whole agent trajectory, not one success bit: did it gather the right evidence, reason from first principles, and justify its decision in an auditable way?
- **AI-assisted grading.** Human review of every trajectory doesn't scale, so calibrate an **LLM judge** (about 100 samples) to apply the same rubric to every run. Weight where humans agree, discount where they disagree. That's how you take the human's bias out.
- **Iterative hill-climbing.** With the eval running, set a deploy bar across those dimensions and push the agent above it.

Reference: [*Joshua Saxe — The Hard Part Isn't Building the Agent: Measuring Effectiveness*](#)

Generate tests together

You can ask AI to generate more tests, and they must be deterministic.

- This is what I did with my outbound-network project: I asked it to list every way a file can be uploaded to a server, and every way a file can be extracted. I don't know every technique of file extraction, but Claude and I can generate meaningful tests together.
- You can also ask Claude to model old cases and incidents as scripts (old breaches or violations become regression tests you re-run forever) to quickly test your pipeline or code.
- Testing is also how you learn that some techniques from others don't work. I once followed a public pentest bot, only to find it wasn't up to my standards. I only caught it after testing showed abysmal recall.

Code Quality

The AI churns out a lot of code. How do you maintain code quality?

Anchor to a known standard

First, ask Claude to output code that matches a known, good coding style. Two I enforce:

- [Google Style Guide](#). Google runs enormous codebases, and a strict shared style is what keeps them easy to review. Borrow that.
- [Trail of Bits' Codex config](#). They condensed good testing practice into a skill you can point the model at and check against (test behavior not implementation, test edges and errors, mock boundaries not logic, verify tests catch failures).

You can take it further: ask an AI agent to review the code and point out mistakes. You'll be surprised how many bugs and inconsistencies it finds in code it just wrote.

AI code review

- Basically after any code changes, you can ask AI to spawn a new agent (that carries no bias or context about the conversation) to adversarially flag out any bugs, inconsistencies or code style issues in your code
- I have some success with using models of different provider (e.g using Codex to review Claude's coding) with good success. We want as many second opinion about the code as possible to fix it

Two rules that save you

- **Always use Git.** Ask the AI to commit after each sizeable feature, so if it does something you didn't want, you can roll back. I did not do this too many times and I always regret when I accidentally delete some code that I spent a whole day on
- **Never point AI at a production database.** I've seen people lose their data because the model misread its own actions. Develop against a test database or environment, never the real one.

Amazon's cloud 'hit by two outages caused by AI tools last year'

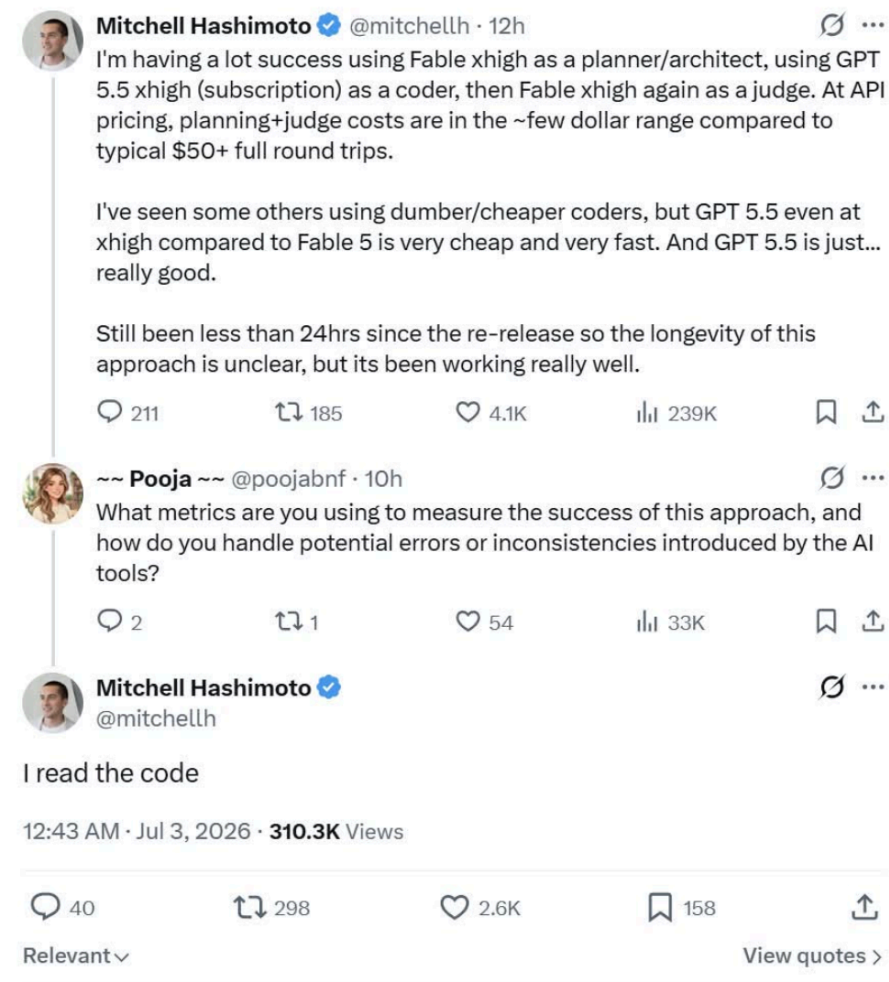
Reported issues at Amazon Web Services raise questions about firm's use of artificial intelligence as it cuts staff

Source: [*The Guardian, AWS outages caused by AI tools*](#)

Where vibe-coding quietly breaks

Reviewing and navigating code still can't be skipped. Flaws I keep hitting:

- **Regex.** AI loves it (Claude especially). In systems work, the appearance of regex is usually bad news. It's hard and error-prone, and often a tell that the model found a cheating shortcut to beat your benchmark. I hate it for exactly that reason.
- **Hardcoding known facts.** As the context window fills, the model "cheats" the implementation. Ask it to detect *all* file-upload cases and it quietly builds for only the case you named.
- **Build small.** For a pipeline $A \rightarrow B \rightarrow C$, build A and evaluate it before B. One giant diff covering A, B, and C is impossible to read.



Can I let the model do everything for me?

Short answer: no. And it's **more dangerous** than it feels.

Don't hand off the thinking

It feels easy to say a few generic words ("build me this pentest application") and walk away, or to click the option the AI marked "recommended." This is **extremely dangerous**.

- **You must own the design.** You must be the one to make the design decision. A complex system can't be described in two or three sentences. If it could, Anthropic would already run the world by issuing 3 sentences
- **Brain rot:** if you become dependent on AI to make decisions, the moment prices climb it will take a long time to re-adapt, if it feels possible at all.
- Real case: Bad actors vibe-coded malware whose C2 address was `localhost`



vx-underground
@vxunderground

```
> get dm
> "is this malware?"
> look inside
> malicious libre office macro
> looks funny tho
> raw shellcode
> 2,0,1,187,192,168,45,246
> malware tries connecting to IP address
> 192.168.45.246:443
> malware delivered from a live website
```

these dumb fucks vibe coded a malware payload and had it connect back to a local ip address. are they actually fucking retarded???

Key point

- Read other resources to sharpen your skill. Being meticulous is hard, but it's the tech debt you have to pay off. Skip it and you ship a mediocre product with mediocre features and weird bugs.
- AI will always agree to whatever that you say, no matter how wrong it is. Without reading and establishing good critical thinking, you will slowly lose the ability to think
- I find that a lot of the times, especially when doing work, AI will omit some parts in your work. Do make sure to re-read carefully as well, like for this presentation I caught it skipping a lot of my analysis for nothing
- **Always doubt the AI's output and never stop asking "dumb" questions inside what it built.** AI will never refuse to answer you. Pay close attention to what it wrote and be firm about your viewpoint, or you'll build a product that leads nowhere.

How to keep up with AI

There are so many trends. Which ones help you keep up fast? I treat it like Google's site-discovery mechanism.

- Start with a few researchers whose work, or reposts, you like.
- Read who they reference, then decide whether to follow those people too.
- A hand-curated set gives you the latest trends at a glance, faster than any traditional news source. I also found that these sources skip the technical details, because it is "too complicated" for the reader
- Social media platforms want to maximize the time you spend on their apps, so spend it on purpose: click less on cat videos and soccer, more on technical information.
- Sidenote: China has excellent researchers, some founding labs or working at big ones like OpenAI. Follow them too.

This is how I stay up to date. There's no secret sauce.

Burnout

- I worked from 10am to 2am the next morning, every day, for the last four weeks working non-stop on AI and doing vibe-coding. The mood swings were real: happy when a test went well, sad when I'd burned six hours watching the AI ignore my design, or when I never truly rested because there were still Claude credits left and one more idea to try. And the low is still there when you keep rediscovering what you built last week, every week.
- Implementation feels like running in circles. Some days you make real progress and think you're almost done; other days you're back where you started a few days ago, relearning the same lessons.
- Vibe-coding is like being addicted to gambling. We're hooked on the sense of success because the AI is a slot machine. My happiest days were the days my quota ran out, because they forced me to do something else. This is the "AI psychosis" that comes from doing too much with AI, or reading too much AI-generated text.

How to navigate burnout

- Force yourself to accept that it's fine to leave some ideas on the table and go outside.
- One upside of AI: you can go for a run or do something else while it works. The downside: that break rarely comes. The next thing the AI does is wrong, or "just one more prompt" pulls you back.
- Give yourself a 1–2 day AI break to freshen up before going back in.

I've found my work efficiency is actually higher on the days I don't touch AI at all.

Last tricks

- **Ask for readable writing.** When the output is hard to read, ask it to rewrite to plain, direct language (for example, Google's technical-writing standards).
- **Persona trick.** AI often performs better with a role to inhabit. I ask it to become an expert security engineer (say, a Google security engineer) to review the code. This still works.
- **AI slide creation:** they're horrible at producing the text you actually want, so ask it to summarize the research, then write the content yourself. It can help with the editing and grammar mistakes

```
History 14/21
) /goal Ok, refactor to clean off the unnatural voice. Some things I want to be fixed, later we can work together, here are my takes first:
- No more of the "out-of-credit.png", this is consuming unnecessary area
- Capture the active voice that I write in my content, don't use the weird AI prose
- Color scheme follows that of the hillhouse.css style
- No need for the introduction part: <section class="slide" data-section="Introduction">, just make it so that the stitle is on the top of the page:
<h2 class="stitle">The frontier is startlingly good.</h2>, push the stitle to the top of the page
- On slide 6, pull the relevant information from the OpenAI blog post about why it is so cheap to run (or make a table on the side to show the
numbering)
- On slide 7, the "my take" part should be omitted, make it so that the image is big enough to see on the screen. For all the slides that follow, make
everything direct. I think closest you can think of is like a Ted Talk style, direct
- On slide 9, annotate the picture to note: from benchmark, Luna (cheapest GPT model) outperforms GLM-5.2 in terms of cost. Also rewrite this part so it
is simple
- On slide 12, the content should not be quoted verbatim like that. Rephrase it to demonstrate the point of "even AI labs now are buying data that is
evaluating hard skills, or skills that require a lot of human judgement"
- On slide 13, annotate the picture with: "here is one attempt from me trying to express my disappointment because Claude did not follow what I write"
```

Thanks

Thanks for joining the session. I hope this was useful.

If you have your own tips and tricks for using AI, please let me know.

References (1/3)

Models & context

- Agents' Last Exam / GPT-5.6: <https://openai.com/index/gpt-5-6/>
- Hugging Face security incident: <https://huggingface.co/blog/security-incident-july-2026>

Working with agents (Thariq's field guide)

- Claude blog: <https://claude.com/blog/a-field-guide-to-claude-fable-finding-your-unknowns>

References (2/3)

X / tweets

- Vikram Aditya (@viks_rum), The goldmine business of selling data to frontier labs: https://x.com/viks_rum/status/2077650169265590727
- Thariq (@trq212), A Field Guide to Fable: <https://x.com/trq212/status/2073100352921215386>
- Lance Martin (@RLanceMartin), Cost effective harnesses with Fable: <https://x.com/RLanceMartin/status/2075641284635799865>
- Shouqiao Wang (@Qiaoqiao2001), research search-strategy prompt (CDC): <https://x.com/Qiaoqiao2001/status/2080003454165295403>
- Uncle Bob Martin (@unclebobmartin), constrain agents with tests, not code review: <https://x.com/unclebobmartin/status/2080257779395154409>

Finding real bugs with AI

- slcyber.io, A \$500k WordPress RCE found with GPT-5.6 Sol Ultra for \$25: <https://slcyber.io/research-center/exploit-brokers-pay-500000-for-a-wordpress-rce-i-found-one-with-gpt5-6/>

References (3/3)

Testing

- OpenAI, Separating signal from noise in coding evaluations: <https://openai.com/index/separating-signal-from-noise-coding-evaluations/>
- Joshua Saxe, Measuring Effectiveness: https://www.youtube.com/watch?v=rO2yA52U_i4

Code quality

- Google Style Guide: <https://google.github.io/styleguide/>
- Trail of Bits Codex config: <https://github.com/trailofbits/codex-config/blob/main/global-agents.md>
- The Guardian, AWS outages caused by AI tools: <https://www.theguardian.com/technology/2026/feb/20/amazon-cloud-outages-ai-tools-amazon-web-services-aws>